PROCESSO SELETIVO

Teste Técnico Remoto

Assistente de Desenvolvimento de Sistemas

Objetivo

Este desafio tem como objetivo avaliar aspectos relacionados à lógica de programação, desenvolvimento em Python, manipulação de dados, tratamento de erros, organização do código e documentação.

A atividade simula uma situação comum do dia a dia da área, envolvendo processamento de dados e automação de tarefas.

Cenário

Uma área da cooperativa disponibiliza diariamente um arquivo contendo solicitações de cadastro em formato JSON.

Seu objetivo é desenvolver uma automação que processe essas informações e gere um arquivo para utilização por outras áreas.

Arquivo de entrada

Utilize o arquivo **solicitacoes.json** fornecido junto com este teste.

O arquivo contém registros com os campos **id**, **nome**, **cpf** e **status**.

EXEMPLO DE REGISTRO

```
[
  {
    "id": 1,
    "nome": "Maria Silva",
    "cpf": "12345678900",
    "status": "APROVADO"
  },
  {
    "id": 2,
    "nome": "Pedro Santos",
    "cpf": "45678912300",
    "status": "PENDENTE"
  }
]
```

Atividade

1. Leia o arquivo JSON

Carregue todos os registros do arquivo de entrada.

2. Filtre os registros

Considere somente solicitações que atendam simultaneamente aos critérios:

- status igual a **APROVADO**;
- CPF não nulo e não vazio.

Não é necessário validar os dígitos verificadores do CPF. Registros inválidos ou incompletos devem ser ignorados e registrados no log, sem interromper todo o processamento.

3. Gere um arquivo CSV

Gere um arquivo chamado **aprovados.csv**, em UTF-8, com as colunas:

```
id,nome,cpf
```

O separador poderá ser vírgula ou ponto e vírgula, desde que essa decisão seja informada no README.

4. Gere um log da execução

Crie um arquivo chamado **processamento.log**, registrando informações relevantes, como:

- início e conclusão do processamento;
- total de registros lidos;
- total de registros exportados;
- registros ignorados e respectivos motivos;
- erros encontrados durante a execução.

O formato do log fica a critério da pessoa candidata.

5. Trate erros

A aplicação deve tratar, no mínimo, as seguintes situações:

- arquivo não encontrado;
- JSON inválido;
- registro sem campos necessários;
- problemas durante a geração do CSV.

O erro deve ser tratado adequadamente e informado no log ou ao usuário.

Entregáveis

Envie um arquivo compactado (**.zip**) contendo:

```
solicitacoes.json
main.py
README.md
requirements.txt (somente se utilizar bibliotecas externas)
outros arquivos necessários, se houver
```

README

O arquivo README deve conter:

- explicação breve da solução;
- pré-requisitos;
- instruções para executar o projeto;
- premissas adotadas;
- decisões relevantes, como o separador utilizado no CSV;
- ferramentas de consulta ou IA utilizadas, quando aplicável.

Integração com processos

Responda brevemente no README:

> Como você adaptaria essa solução para um processo de BPM, no qual os registros fossem recebidos por um formulário e o arquivo CSV fosse gerado após uma etapa de aprovação?

Regras

- A solução deve ser desenvolvida em Python.
- Bibliotecas nativas podem ser utilizadas livremente.
- Bibliotecas externas podem ser utilizadas, desde que informadas no README.
- O código deve estar funcional e executável.
- É permitido consultar documentação, fóruns, mecanismos de busca e ferramentas de inteligência artificial.

> **Transparência no uso de ferramentas**
>
> Caso utilize alguma ferramenta, informe qual foi utilizada, em que etapa ela contribuiu e quais decisões ou alterações foram feitas por você. Durante a entrevista, algumas respostas poderão ser discutidas para verificar seu entendimento.

Critérios de avaliação

Critério	Pontuação
Atendimento aos requisitos	25
Lógica e funcionamento da solução	25
Organização e legibilidade do código	15
Tratamento de erros e dados inválidos	20
Documentação e instruções de execução	10
Clareza das premissas e decisões	5
Total	**100**

Diferenciais

Serão considerados diferenciais:

- uso de Git;
- estruturação do código em funções;
- validações adicionais de dados;
- logs claros e completos;
- testes automatizados;
- organização e manutenibilidade do projeto.

> **Observação:** não buscamos uma solução perfeita ou complexa. O objetivo é compreender como a pessoa candidata interpreta requisitos, organiza a implementação, resolve problemas e documenta sua solução.